Nano-vLLM 내부 동작 과정 상세 분석

목차

1. 개요
2. 전체 아키텍처
3. 초기화 과정
4. 메모리 관리 시스템
5. 스케줄링 시스템
6. 모델 실행 과정
7. 최적화 기법
8. 성능 분석

개요

Nano-vLLM은 vLLM의 핵심 기능을 약 1,200줄의 Python 코드로 재구현한 경량화된 LLM 추론 엔진입니다. 원본 vLLM과 비교해 5% 더 빠른 성능을 달성하면서도 코드베이스를 대폭 단순화했습니다.

핵심 설계 원칙

- **단순성**: 복잡한 기능을 제거하고 핵심 기능에 집중
- **성능**: 메모리 효율성과 처리 속도 최적화
- **가독성**: 이해하기 쉬운 코드 구조

전체 아키텍처

```
┌─────────────────┐      ┌─────────────────┐     ┌─────────────────┐
|    LLM (API)    |      |    LLMEngine    |     |    ModelRunner  |
|                 |──▶|                 |──▶|                 |
|   - generate()  |      | - add_request() |     | - run()         |
|- 사용자 인터페이스|     | - step()        |     | - 모델 실행      |
└─────────────────┘      └─────────────────┘     └─────────────────┘
                                  |                     |
                                  ▼                     ▼
                         ┌─────────────────┐     ┌─────────────────┐
                         |    Scheduler    |     |   BlockManager  |
                         |                 |     |                 |
                         | - schedule()    |     | - allocate()    |
                         | - 배치 관리      |     | - KV 캐시 관리    |
                         └─────────────────┘     └─────────────────┘
                                  |
                                  ▼
                          ┌─────────────────┐
                          |     Sequence    |
                          |                 |
                          | - 개별 요청 상태 |
                          | - 토큰 관리      |
                          └─────────────────┘
```

각 컴포넌트의 역할

1. **LLM**: 사용자 API 인터페이스
2. **LLMEngine**: 메인 추론 엔진, 전체 워크플로우 조정
3. **Scheduler**: 요청 스케줄링 및 배치 관리
4. **ModelRunner**: 실제 모델 실행 및 GPU 최적화
5. **BlockManager**: KV 캐시 메모리 관리
6. **Sequence**: 개별 요청의 상태 및 데이터 관리

초기화 과정

1. LLMEngine 초기화

```
def __init__(self, model, **kwargs):
# 1. 설정 파싱
config_fields = {field.name for field in fields(Config)}
config_kwargs = {k: v for k, v in kwargs.items() if k in config_fields}
config = Config(model, **config_kwargs)
```

왜 이렇게 하는가?

- 사용자가 전달한 매개변수 중 Config 클래스에 정의된 것만 필터링
- 잘못된 매개변수로 인한 오류 방지
- 타입 안전성 보장

2. 멀티프로세스 설정

```
# 텐서 병렬 처리를 위한 프로세스 생성
ctx = mp.get_context("spawn")
for i in range(1, config.tensor_parallel_size):
event = ctx.Event()
process = ctx.Process(target=ModelRunner, args=(config, i, event))
process.start()
self.ps.append(process)
self.events.append(event)
```

왜 spawn 컨텍스트를 사용하는가?

- CUDA 컨텍스트는 프로세스 간 공유가 불가능
- `spawn` 은 완전히 새로운 Python 인터프리터를 시작하여 CUDA 초기화 문제 해결
- `fork` 는 Linux에서만 사용 가능하고 CUDA와 호환성 문제 존재

3. ModelRunner 초기화 상세

3.1 분산 처리 설정

```
dist.init_process_group("nccl", "tcp://localhost:2333",
world_size=self.world_size, rank=rank)
torch.cuda.set_device(rank)
```

NCCL을 사용하는 이유:

- GPU 간 고성능 통신을 위한 NVIDIA의 최적화된 라이브러리
- AllReduce, AllGather 등의 집합 통신 연산 지원

- InfiniBand, Ethernet 등 다양한 네트워크 지원

3.2 메모리 워밍업

```
def warmup_model(self):
torch.cuda.empty_cache()
torch.cuda.reset_peak_memory_stats()
max_num_batched_tokens, max_model_len = self.config.max_num_batched_tokens, self
num_seqs = min(max_num_batched_tokens // max_model_len, self.config.max_num_seqs
seqs = [Sequence([0] * max_model_len) for _ in range(num_seqs)]
self.run(seqs, True)
torch.cuda.empty_cache()
```

워밍업이 필요한 이유:

- PyTorch는 lazy allocation을 사용하여 실제 사용 시점에 메모리 할당
- 최대 메모리 사용량을 미리 측정하여 KV 캐시 크기 계산
- CUDA 커널 컴파일 및 최적화 수행

3.3 KV 캐시 메모리 할당

```
def allocate_kv_cache(self):
free, total = torch.cuda.mem_get_info()
used = total - free
peak = torch.cuda.memory_stats()["allocated_bytes.all.peak"]
current = torch.cuda.memory_stats()["allocated_bytes.all.current"]
```

```
num_kv_heads = hf_config.num_key_value_heads // self.world_size

block_bytes = 2 * hf_config.num_hidden_layers * self.block_size * num_kv_heads *

config.num_kvcache_blocks = int(total * config.gpu_memory_utilization - used - p
```

메모리 계산 공식 설명:

- `2` : Key와 Value 캐시 각각 하나씩
- `num_hidden_layers` : 트랜스포머의 레이어 수
- `block_size` : 블록당 토큰 수 (기본 256)
- `num_kv_heads` : KV 헤드 수 (텐서 병렬로 분할)
- `head_dim` : 헤드 차원
- `torch_dtype.itemsize` : 데이터 타입별 바이트 크기

왜 블록 단위로 관리하는가?

- 메모리 단편화 방지
- 효율적인 할당/해제
- Prefix 캐싱 지원

메모리 관리 시스템

BlockManager의 핵심 개념

1. 블록 구조

```
class Block:

def __init__(self, block_id):

self.block_id = block_id # 블록 식별자
```

```
self.ref_count = 0 # 참조 카운트

self.hash = -1 # 블록 해시값

self.token_ids = [] # 저장된 토큰들
```

2. Prefix 캐싱 메커니즘

```
def allocate(self, seq: Sequence):

h = -1

cache_miss = False

for i in range(seq.num_blocks):

token_ids = seq.block(i)

# 체인 해싱: 이전 블록의 해시를 포함하여 계산

h = self.compute_hash(token_ids, h) if len(token_ids) == block_size else -1

block_id = self.hash_to_block_id.get(h, -1)

# 캐시 히트 확인

if block_id == -1 or self.blocks[block_id].token_ids != token_ids:

cache_miss = True

if cache_miss:

# 새 블록 할당

block_id = self.free_block_ids[0]

block = self._allocate_block(block_id)

else:

# 캐시된 블록 재사용
```

```
seq.num_cached_tokens += self.block_size

if block_id in self.used_block_ids:

block = self.blocks[block_id]

block.ref_count += 1

else:

block = self._allocate_block(block_id)
```

체인 해싱을 사용하는 이유:

- 단순 해시는 동일한 블록이 다른 위치에서 다른 의미를 가질 수 있음
- 이전 블록의 컨텍스트를 포함하여 정확한 매칭 보장
- 예: "Hello world"와 "world Hello"에서 “world” 블록을 구분

3. 참조 카운팅

```
def deallocate(self, seq: Sequence):

for block_id in reversed(seq.block_table):

block = self.blocks[block_id]

block.ref_count -= 1

if block.ref_count == 0:

self._deallocate_block(block_id)
```

참조 카운팅의 장점:

- 여러 시퀀스가 동일한 블록을 공유 가능
- 자동 메모리 해제
- 메모리 누수 방지

스케줄링 시스템

Scheduler의 두 단계 처리

1. Prefill 단계

```
# prefill 스케줄링
scheduled_seqs = []
num_seqs = 0
num_batched_tokens = 0

while self.waiting and num_seqs < self.max_num_seqs:
seq = self.waiting[0]
if (num_batched_tokens + len(seq) > self.max_num_batched_tokens or
not self.block_manager.can_allocate(seq)):
break
num_seqs += 1
self.block_manager.allocate(seq)
num_batched_tokens += len(seq) - seq.num_cached_tokens
seq.status = SequenceStatus.RUNNING
self.waiting.popleft()
self.running.append(seq)
scheduled_seqs.append(seq)
```

```
if scheduled_seqs:
return scheduled_seqs, True # is_prefill = True
```

Prefill 단계의 특징:

- 전체 프롬프트를 한 번에 처리
- 토큰 수가 많아 메모리 사용량이 큼
- 병렬 처리 효율성이 높음
- `num_batched_tokens` 제한으로 OOM 방지

2. Decode 단계

```
# decode 스케줄링
while self.running and num_seqs < self.max_num_seqs:
seq = self.running.popleft()
# 메모리 부족 시 preemption
while not self.block_manager.can_append(seq):
if self.running:
self.preempt(self.running.pop())
else:
self.preempt(seq)
break
else:
num_seqs += 1
self.block_manager.may_append(seq)
scheduled_seqs.append(seq)
```

```
assert scheduled_seqs
self.running.extendleft(reversed(scheduled_seqs))
return scheduled_seqs, False # is_prefill = False
```

Decode 단계의 특징:

- 한 번에 하나의 토큰만 생성
- 시퀀스 수가 많을수록 처리량 증가
- 메모리 사용량이 상대적으로 적음
- Preemption을 통한 동적 메모리 관리

3. Preemption 메커니즘

```
def preempt(self, seq: Sequence):
seq.status = SequenceStatus.WAITING
self.block_manager.deallocate(seq)
self.waiting.appendleft(seq)
```

Preemption이 필요한 이유:

- 메모리 부족 시 일부 시퀀스를 일시 중단
- 높은 우선순위 요청을 위한 자원 확보
- 시스템 안정성 보장

스케줄링 정책

FCFS (First Come First Served) 기반:

- 공정성 보장
- 구현 단순성
- 예측 가능한 동작

향후 개선 가능한 정책들:

- Priority-based scheduling
- Shortest Job First (SJF)
- Round Robin with time slicing

모델 실행 과정

1. 입력 데이터 준비

Prefill 데이터 준비

```
def prepare_prefill(self, seqs: list[Sequence]):
input_ids = []
positions = []
cu_seqlens_q = [0] # cumulative sequence lengths for query
cu_seqlens_k = [0] # cumulative sequence lengths for key
max_seqlen_q = 0
max_seqlen_k = 0
slot_mapping = []
for seq in seqs:
seqlen = len(seq)
# 캐시되지 않은 토큰들만 처리
```

```
input_ids.extend(seq[seq.num_cached_tokens:])

positions.extend(list(range(seq.num_cached_tokens, seqlen)))

seqlen_q = seqlen - seq.num_cached_tokens # 새로 계산할 토큰 수

seqlen_k = seqlen # 전체 컨텍스트 길이

cu_seqlens_q.append(cu_seqlens_q[-1] + seqlen_q)

cu_seqlens_k.append(cu_seqlens_k[-1] + seqlen_k)

max_seqlen_q = max(seqlen_q, max_seqlen_q)

max_seqlen_k = max(seqlen_k, max_seqlen_k)

# KV 캐시 슬롯 매핑 계산

if seq.block_table:

for i in range(seq.num_cached_blocks, seq.num_blocks):

start = seq.block_table[i] * self.block_size

if i != seq.num_blocks - 1:

end = start + self.block_size

else:

end = start + seq.last_block_num_tokens

slot_mapping.extend(list(range(start, end)))
```

데이터 구조 설명:

- `cu_seqlens_q/k` : Flash Attention의 variable length 처리를 위한 누적 길이
- `slot_mapping` : 각 토큰이 KV 캐시의 어느 위치에 저장될지 매핑
- `positions` : 각 토큰의 위치 인코딩

Decode 데이터 준비

```
def prepare_decode(self, seqs: list[Sequence]):

input_ids = []

positions = []

slot_mapping = []

context_lens = []

for seq in seqs:

input_ids.append(seq.last_token) # 마지막 토큰만

positions.append(len(seq)) # 현재 위치

context_lens.append(len(seq)) # 컨텍스트 길이

# 새 토큰이 저장될 KV 캐시 위치

slot_mapping.append(seq.block_table[-1] * self.block_size + seq.last_block_num_t
```

2. 어텐션 계산

Flash Attention 사용

```
def forward(self, q: torch.Tensor, k: torch.Tensor, v: torch.Tensor):

q = q.view(-1, self.num_heads, self.head_dim)

k = k.view(-1, self.num_kv_heads, self.head_dim)

v = v.view(-1, self.num_kv_heads, self.head_dim)

context = get_context()

k_cache, v_cache = self.k_cache, self.v_cache

# KV 캐시에 저장 (Triton 커널 사용)
```

```
if k_cache.numel() and v_cache.numel():
store_kvcache(k, v, k_cache, v_cache, context.slot_mapping)
if context.is_prefill:
if context.block_tables is not None: # prefix cache 사용
k, v = k_cache, v_cache
# Variable length Flash Attention
o = flash_attn_varlen_func(q, k, v,
max_seqlen_q=context.max_seqlen_q,
cu_seqlens_q=context.cu_seqlens_q,
max_seqlen_k=context.max_seqlen_k,
cu_seqlens_k=context.cu_seqlens_k,
softmax_scale=self.scale, causal=True,
block_table=context.block_tables)
else: # decode
# KV 캐시와 함께 Flash Attention
o = flash_attn_with_kvcache(q.unsqueeze(1), k_cache, v_cache,
cache_seqlens=context.context_lens,
block_table=context.block_tables,
softmax_scale=self.scale, causal=True)
return o.view(-1, self.num_heads * self.head_dim)
```

Flash Attention의 장점:

- 메모리 사용량 $O(N^2) \rightarrow O(N)$으로 감소

- 타일링을 통한 효율적인 메모리 접근
- 하드웨어 최적화된 구현

Triton 커널을 사용한 KV 캐시 저장

```
@triton.jit
def store_kvcache_kernel(
key_ptr, key_stride, value_ptr, value_stride,
k_cache_ptr, v_cache_ptr, slot_mapping_ptr,
D: tl.constexpr,
):
idx = tl.program_id(0)
key_offsets = idx * key_stride + tl.arange(0, D)
value_offsets = idx * value_stride + tl.arange(0, D)
key = tl.load(key_ptr + key_offsets)
value = tl.load(value_ptr + value_offsets)
slot = tl.load(slot_mapping_ptr + idx)
cache_offsets = slot * D + tl.arange(0, D)
tl.store(k_cache_ptr + cache_offsets, key)
tl.store(v_cache_ptr + cache_offsets, value)
```

Triton 사용 이유:

- CUDA 커널보다 작성이 쉬움
- 자동 최적화 및 튜닝

- 메모리 접근 패턴 최적화

3. 샘플링

Gumbel 샘플링 구현

```
def forward(self, logits: torch.Tensor, temperatures: torch.Tensor):

logits = logits.to(torch.float)

greedy_tokens = logits.argmax(dim=-1)

# 온도 스케일링

logits.div_(temperatures.unsqueeze(dim=1))

probs = torch.softmax(logits, dim=-1, dtype=torch.float)

# Gumbel 샘플링: G = -log(-log(U))에서 U는 uniform(0,1)

# 하지만 exponential(1) 분포를 직접 사용하여 더 효율적으로 구현

epsilon = 1e-10

sample_tokens = probs.div_(torch.empty_like(probs).exponential_(1) + epsilon).ar

return torch.where(temperatures == 0, greedy_tokens, sample_tokens)
```

Gumbel 샘플링의 장점:

- 수치적으로 안정적
- 벡터화된 구현 가능
- Top-k, Top-p 등 다양한 샘플링 전략과 결합 가능

최적화 기법

1. CUDA 그래프 최적화

그래프 캡처 과정

```
def capture_cudagraph(self):
max_bs = min(self.config.max_num_seqs, 512)
# 다양한 배치 크기별로 그래프 캡처
self.graph_bs = [1, 2, 4, 8] + list(range(16, max_bs + 1, 16))
self.graphs = {}
self.graph_pool = None
for bs in reversed(self.graph_bs):
graph = torch.cuda.CUDAGraph()
# 컨텍스트 설정
set_context(False, slot_mapping=slot_mapping[:bs],
context_lens=context_lens[:bs],
block_tables=block_tables[:bs])
# 워밍업 실행
outputs[:bs] = self.model(input_ids[:bs], positions[:bs])
# 그래프 캡처
with torch.cuda.graph(graph, self.graph_pool):
outputs[:bs] = self.model(input_ids[:bs], positions[:bs])
if self.graph_pool is None:
self.graph_pool = graph.pool()
self.graphs[bs] = graph
```

CUDA 그래프의 장점:

- 커널 런치 오버헤드 제거 (CPU-GPU 동기화 최소화)
- 작은 배치에서 특히 효과적
- 메모리 할당 오버헤드 제거

왜 512 이하에서만 사용하는가?

- 큰 배치에서는 커널 실행 시간이 오버헤드보다 훨씬 큼
- 메모리 사용량 증가
- 그래프 캡처 시간 증가

그래프 실행

```
def run_model(self, input_ids, positions, is_prefill):
if is_prefill or self.enforce_eager or input_ids.size(0) > 512:
# 일반 실행
return self.model.compute_logits(self.model(input_ids, positions))
else:
# CUDA 그래프 사용
bs = input_ids.size(0)
graph = self.graphs[next(x for x in self.graph_bs if x >= bs)]
# 입력 데이터 복사
graph_vars = self.graph_vars
for k, v in graph_vars.items():
if k != "outputs":
v.zero_()
graph_vars["input_ids"][:bs] = input_ids
```

```
graph_vars["positions"][:bs] = positions

# ... 기타 변수들

# 그래프 재실행

graph.replay()

return self.model.compute_logits(graph_vars["outputs"][:bs])
```

2. 텐서 병렬 처리

프로세스 간 통신

```
def write_shm(self, method_name, *args):

# 공유 메모리를 통한 데이터 전달

data = pickle.dumps([method_name, *args])

n = len(data)

self.shm.buf[0:4] = n.to_bytes(4, "little")

self.shm.buf[4:n+4] = data

# 모든 워커 프로세스에 신호 전송

for event in self.event:

event.set()
```

공유 메모리 사용 이유:

- 프로세스 간 대용량 데이터 전달 시 복사 오버헤드 최소화
- 직렬화/역직렬화 오버헤드는 있지만 네트워크 통신보다 빠름
- 단일 노드 내에서 효율적

3. 메모리 최적화

동적 메모리 할당

```
def allocate_kv_cache(self):
# 실제 사용 가능한 메모리 계산
free, total = torch.cuda.mem_get_info()
used = total - free
peak = torch.cuda.memory_stats()["allocated_bytes.all.peak"]
current = torch.cuda.memory_stats()["allocated_bytes.all.current"]
# 안전 마진을 고려한 메모리 할당
available_memory = total * config.gpu_memory_utilization - used - peak + current
config.num_kvcache_blocks = available_memory // block_bytes
```

동적 할당의 장점:

- 하드웨어별 최적화
- 메모리 사용률 극대화
- OOM 오류 방지

성능 분석

벤치마크 결과 분석

| 메트릭 | vLLM | Nano-vLLM | 개선율 |

|--------|------|-----------|--------|

| 처리량 | 1361.84 tok/s | 1434.13 tok/s | +5.3% |

| 총 시간 | 98.37s | 93.41s | -5.0% |

| 코드 라인 수 | ~50,000 | ~1,200 | -97.6% |

성능 향상 요인

1. **코드 단순화**

- 불필요한 추상화 제거
- 직접적인 구현으로 오버헤드 감소

2. **최적화된 메모리 관리**

- 효율적인 블록 관리
- Prefix 캐싱 최적화

3. **CUDA 그래프 활용**

- 작은 배치에서 큰 성능 향상
- 커널 런치 오버헤드 제거

4. **Flash Attention 통합**

- 메모리 효율적인 어텐션 계산
- 하드웨어 최적화

메모리 사용량 분석

```
총 GPU 메모리: 8GB

├── 모델 가중치: ~1.2GB (Qwen3-0.6B)

├── KV 캐시: ~5.5GB (gpu_memory_utilization * 0.9)

├── 활성화 메모리: ~1.0GB
```

```
└── 시스템 예약: ~0.3GB
```

메모리 효율성:

- KV 캐시가 전체 메모리의 약 70% 사용
- 블록 단위 관리로 단편화 최소화
- Prefix 캐싱으로 중복 제거

처리량 특성

Prefill vs Decode 성능

- **Prefill**: 높은 메모리 대역폭 활용, 병렬 처리 효율성
- **Decode**: 낮은 지연시간, 높은 배치 처리량

배치 크기별 성능

- 작은 배치 (1-8): CUDA 그래프로 큰 성능 향상
- 중간 배치 (16-64): 최적의 처리량/지연시간 균형
- 큰 배치 (128+): 메모리 제약으로 성능 저하

결론

Nano-vLLM은 다음과 같은 핵심 설계 원칙을 통해 높은 성능을 달성합니다:

1. **단순성**: 복잡한 기능을 제거하고 핵심에 집중
2. **효율성**: 메모리와 계산 자원의 최적 활용
3. **최적화**: 하드웨어 특성을 고려한 구현

이러한 접근 방식은 LLM 추론 엔진 설계에서 "적은 것이 더 많은 것"이라는 철학을 잘 보여주는 사례입니다.

부록:

코드 흐름 다이어그램

전체 실행 흐름

```
graph TD

A[사용자 요청] --> B[LLM.generate()]

B --> C[LLMEngine.add_request()]

C --> D[Sequence 생성]

D --> E[Scheduler.add()]

E --> F{모든 요청 완료?}

F -->|No| G[Scheduler.schedule()]

G --> H{Prefill vs Decode?}

H -->|Prefill| I[prepare_prefill()]

H -->|Decode| J[prepare_decode()]

I --> K[ModelRunner.run()]

J --> K

K --> L[Flash Attention]

L --> M[Sampler]

M --> N[Scheduler.postprocess()]

N --> O[토큰 추가]

O --> F
```

```
F -->|Yes| P[결과 반환]
```

메모리 관리 흐름

```
graph TD

A[Sequence 생성] --> B[BlockManager.can_allocate()]

B -->|Yes| C[BlockManager.allocate()]

B -->|No| D[대기 큐에 유지]

C --> E{Prefix 캐시 히트?}

E -->|Yes| F[기존 블록 재사용]

E -->|No| G[새 블록 할당]

F --> H[ref_count 증가]

G --> I[블록 해시 계산]

H --> J[KV 캐시 슬롯 매핑]

I --> J

J --> K[모델 실행]

K --> L[Sequence 완료?]

L -->|Yes| M[BlockManager.deallocate()]

L -->|No| N[다음 토큰 생성]

M --> O[ref_count 감소]

O --> P{ref_count == 0?}

P -->|Yes| Q[블록 해제]

P -->|No| R[블록 유지]
```

CUDA 그래프 실행 흐름

```
graph TD

A[모델 실행 요청] --> B{배치 크기 <= 512?}

B -->|No| C[일반 실행]

B -->|Yes| D{Prefill?}

D -->|Yes| C

D -->|No| E[적절한 그래프 선택]

E --> F[그래프 변수 업데이트]

F --> G[graph.replay()]

G --> H[결과 추출]

C --> I[model.forward()]

I --> H

H --> J[샘플링]
```

부록: 성능 튜닝 가이드

1. 메모리 최적화

GPU 메모리 사용률 조정

```
# 메모리 사용률을 높여 더 많은 KV 캐시 확보

llm = LLM(model_path, gpu_memory_utilization=0.95)
```

```
# 메모리 부족 시 사용률 낮춤
llm = LLM(model_path, gpu_memory_utilization=0.8)
```

블록 크기 조정

```
# 더 큰 블록 크기로 메모리 오버헤드 감소 (256의 배수여야 함)
llm = LLM(model_path, kvcache_block_size=512)
```

2. 배치 처리 최적화

최대 배치 토큰 수 조정

```
# 더 큰 배치 처리 (메모리 허용 시)
llm = LLM(model_path, max_num_batched_tokens=32768)

# 메모리 부족 시 배치 크기 감소
llm = LLM(model_path, max_num_batched_tokens=8192)
```

최대 시퀀스 수 조정

```
# 더 많은 동시 요청 처리
llm = LLM(model_path, max_num_seqs=1024)
```

3. 텐서 병렬 처리

다중 GPU 사용

```
# 2개 GPU 사용
llm = LLM(model_path, tensor_parallel_size=2)


# 4개 GPU 사용 (큰 모델에 효과적)
llm = LLM(model_path, tensor_parallel_size=4)
```

4. CUDA 그래프 제어

CUDA 그래프 비활성화

```
# 디버깅이나 특수한 경우에 사용
llm = LLM(model_path, enforce_eager=True)
```

5. 샘플링 매개변수 최적화

처리량 우선 설정

```
# 낮은 온도로 더 예측 가능한 출력
sampling_params = SamplingParams(temperature=0.1, max_tokens=128)
```

품질 우선 설정

```
# 높은 온도로 더 창의적인 출력
sampling_params = SamplingParams(temperature=0.8, max_tokens=512)
```

부록: 문제 해결 가이드

1. 메모리 부족 오류

증상: `CUDA out of memory` 오류

해결책:

```
# 1. GPU 메모리 사용률 감소
llm = LLM(model_path, gpu_memory_utilization=0.7)

# 2. 배치 크기 감소
llm = LLM(model_path, max_num_batched_tokens=8192)

# 3. 최대 시퀀스 수 감소
llm = LLM(model_path, max_num_seqs=256)
```

2. 낮은 처리량

증상: 예상보다 느린 추론 속도

해결책:

```
# 1. CUDA 그래프 활성화 확인
llm = LLM(model_path, enforce_eager=False)

# 2. 배치 크기 증가
llm = LLM(model_path, max_num_batched_tokens=32768)
```

```
# 3. 텐서 병렬 처리 사용
llm = LLM(model_path, tensor_parallel_size=2)
```

3. 높은 지연시간

증상: 첫 번째 토큰 생성이 느림

해결책:

```
# 1. Prefill 배치 크기 제한
llm = LLM(model_path, max_num_batched_tokens=16384)

# 2. 더 작은 블록 크기 사용
llm = LLM(model_path, kvcache_block_size=256)
```

4. 분산 처리 오류

증상: 멀티 GPU 설정에서 오류 발생

해결책:

```
# 1. NCCL 환경 변수 설정
export NCCL_DEBUG=INFO
export NCCL_IB_DISABLE=1 # InfiniBand 비활성화

# 2. 포트 충돌 확인
```

```
netstat -an | grep 2333
```

부록: 확장 가능성

1. 새로운 모델 지원

현재 Qwen3만 지원하지만, 다른 모델 추가 가능:

```
# nanovllm/models/llama.py
class LlamaForCausalLM(nn.Module):
def __init__(self, config):
# Llama 모델 구현
pass
```

2. 새로운 샘플링 전략

```
# nanovllm/layers/sampler.py
class AdvancedSampler(Sampler):
def forward(self, logits, sampling_params):
# Top-k, Top-p, Typical sampling 등 구현
pass
```

3. 동적 배치 크기

```
# 요청 패턴에 따른 동적 배치 크기 조정

class AdaptiveScheduler(Scheduler):

def schedule(self):

# 동적 배치 크기 결정 로직

pass
```

4. 모델 양자화 지원

```
# INT8, FP16 양자화 지원

class QuantizedModelRunner(ModelRunner):

def __init__(self, config):

# 양자화된 모델 로딩

pass
```

이러한 확장을 통해 Nano-vLLM은 더 다양한 사용 사례와 하드웨어 환경을 지원할 수 있습니다.